

# West, Topologies & Hello World

Zephyr RTOS Course





## Quick Recap — L01



- What Zephyr is: RTOS by Linux Foundation, multi-arch, modular
- Zephyr vs FreeRTOS: different trade-offs, not better/worse
- What's inside: drivers, networking, 3rd party, subsystems
- Environment setup: dependencies, `venv`, `west`, SDK
- Built Hello World from `zephyr/samples/hello_world`



## Format



- Lecture with optional **live demo**
- Homework after each session (checked by us)
- Recordings will be shared
- Questions addressed at the end in a **Q&A session**



## Today's Agenda



- Meet `west` — Zephyr's meta-tool
- **Project topologies** — freestanding, repository, workspace
- The `west.yml` manifest
- Creating a **workspace from scratch**
- **Hello World** walkthrough — code, build, flash
- Troubleshooting tips
- Tasks



Meet west



## Zephyr's Meta-Tool



- **Python-based** build orchestrator
- Handles:
  - **Repository management** (multi-module workspaces)
  - **Build system** integration (CMake + Ninja)
  - **Flashing**, debugging, logging
- Wraps `CMake` + `ninja` for convenience
- Extensible with custom commands



## Key Subcommands



| Command                         | Purpose                                |
|---------------------------------|----------------------------------------|
| <code>west init</code>          | Initialize a workspace                 |
| <code>west update</code>        | Fetch/update all manifest dependencies |
| <code>west build</code>         | Configure and build an application     |
| <code>west flash</code>         | Flash the built image to a board       |
| <code>west debug</code>         | Launch a debugger session              |
| <code>west boards</code>        | List all supported boards              |
| <code>west zephyr-export</code> | Register Zephyr CMake package          |



# Project Topologies



# Freestanding Application



```
<home>/
├── zephyrproject/
│   ├── .west/
│   │   └── config
│   ├── zephyr/
│   └── ...
└── my-app/
    ├── CMakeLists.txt
    ├── prj.conf
    └── src/
        └── main.c
```



# Repository Application



```
zephyrproject/  
├── .west/  
│   └── config  
└── zephyr/  
    ├── arch/  
    ├── boards/  
    ├── cmake/  
    ├── samples/  
    │   ├── hello_world/  
    │   └── ...  
    ├── tests/  
    └── ...
```



## Workspace Application



```
my-project/
├── .west/
│   └── config
├── app/
│   ├── CMakeLists.txt
│   ├── prj.conf
│   ├── west.yml
│   └── src/
│       └── main.c
├── zephyr/
├── bootloader/
├── modules/
└── <vendor/private-repos>/
```



## Which Topology to Use?



| Topology            | When to use                                       |
|---------------------|---------------------------------------------------|
| <b>Freestanding</b> | Quick experiments, learning, simple projects      |
| <b>Repository</b>   | Contributing to Zephyr samples/tests              |
| <b>Workspace</b>    | Production projects, custom boards, multiple apps |

**Recommendation:** Use **Workspace** for real projects



# The `west.yml` Manifest



west.yml



```
manifest:
  version: 1.0                                # manifest schema version
  projects:
    - name: zephyr                            # dependency name
      url: https://github.com/zephyrproject-rtos/zephyr
      revision: v4.2.0                        # pinned git tag
      import:
        name-allowlist:                       # only fetch what we need
          - cmsis
          - hal_nxp
        path-prefix: deps                    # put deps in deps/
```

- Omit `name-allowlist` to fetch all Zephyr sub-projects (~80 repos)



# Creating a Workspace



## Step 1: Initialize



```
# From a local directory
git clone https://github.com/iomicro-public/zephyr-course
cd zephyr-course
west init -l

# Or from a remote repository
west init -m https://github.com/iomicro-public/zephyr-course
```



## Step 2: Fetch Dependencies



```
west update
```

This downloads:

- Zephyr (at the pinned revision)
- All allowlisted sub-projects (cmsis, hal\_nxp, etc.)



## Step 3: Set Up Environment



```
# Register Zephyr CMake package  
west zephyr-export
```



## Resulting Structure



```
workspace/
├── .west/
│   └── config
├── zephyr-course/
│   ├── west.yml
│   └── app
│       ├── CMakeLists.txt
│       ├── prj.conf
│       └── src/
│           └── main.c
└── deps/
    ├── zephyr/
    ├── modules/
    │   ├── hal/
    │   └── nxp/
    └── ...
```



# Hello World Walkthrough



main.c



```
#include <zephyr/kernel.h>

int main(void)
{
    printk("Hello World! %s\n", CONFIG_BOARD_TARGET);
    return 0;
}
```



CMakeLists.txt



```
cmake_minimum_required(VERSION 3.20.0)

find_package(Zephyr REQUIRED)
project(my_zephyr_app)

target_sources(app PRIVATE src/main.c)
```



prj.conf



```
# Project-specific Kconfig options go here  
# Example: CONFIG_GPIO=y
```

- Kconfig configuration file for your application
- Can be **empty** for minimal applications
- Details in **L03** (KConfig & Devicetree)



Build, Flash, Run



Building with `west`



```
west build --board frdm_rw612/rw612 app
```

Or without west:

```
cmake -B build -DBOARD=frdm_rw612/rw612 -G Ninja -S app  
cmake --build build
```



## Flashing and Running



```
# Flash to hardware
west flash

# Debug with GDB
west debug

# Run on native_sim (no hardware needed)
west build --board native_sim app
west build -t run
```



## LED Blink Example



```
#include <zephyr/drivers/gpio.h>

static const struct gpio_dt_spec led =
    GPIO_DT_SPEC_GET(DT_ALIAS(led0), gpios);

int main(void)
{
    if (!gpio_is_ready_dt(&led)) return 0;
    gpio_pin_configure_dt(&led, GPIO_OUTPUT_ACTIVE);

    while (1) {
        gpio_pin_toggle_dt(&led);
        k_msleep(1000);
    }
}
```



# Troubleshooting



## Common Issues



| Problem                                 | Cause                        | Fix                                       |
|-----------------------------------------|------------------------------|-------------------------------------------|
| <code>find_package(Zephyr)</code> fails | Missing env setup            | Run <code>west zephyr-export</code>       |
| Build fails with missing board          | Wrong board name             | Check <code>west boards</code>            |
| Missing HAL errors                      | Incomplete manifest          | Add module to <code>name-allowlist</code> |
| SDK version mismatch                    | Wrong SDK for Zephyr version | Check compatibility table                 |



## SDK Version Compatibility



- Each Zephyr release specifies a **minimum SDK version**
- Check: [Zephyr SDK Toolchain docs](#)
- When upgrading Zephyr, always verify SDK compatibility
- SDK can usually be updated independently



Tasks



0. **Read:** [West documentation](#)

1. **Fork and set up the provided workspace** — <https://github.com/iomico-public/zephyr-course>

2. **Build and run an LED blink application**

- Flash to your hardware board
- Verify the LED toggles every second (if there's an LED)
- Push tag: `l2-task1`



## Homework (cont.)



### 3. (Optional) Build and run on `native_sim`

- Check the boot header for the OS version

```
*** Booting Zephyr OS build v4.2.0 ***
```

**Thank You!**





## Resources



1. [West \(Zephyr's meta-tool\) documentation](#)
2. [West manifest format](#)
3. [Zephyr application development](#)
4. [Zephyr SDK Toolchain docs](#)
5. [Supported boards](#)